

Algoritmos y Estructura de Datos

Estructura de Datos Dinamicas



Universidad
Tecnológica
del Perú

Introducción

“Si quieres
llegar *rápido*,
camina *solo*.
Si quieres llegar *lejos*,
camina en *grupo*”.

Proverbio Africano

Recordando

- Las operaciones con arreglos lineales son
- Las operación con Arreglos Bidimensionales son

Desaprende lo que te limita

Agenda

- **ESTRUCTURAS DE DATOS DINÁMICAS:**
- **Tipo Abstracto de Datos (TAD).**
Conceptos, Especificación informal de un TAD
Especificación formal de un TAD. Implementación del TAD
- Ejercicios
- Resumen

Desaprende lo que te limita

**CONOCIMIENTOS ADQUIRIDOS EN
LA PLATAFORMA SOBRE EL TEMA.**

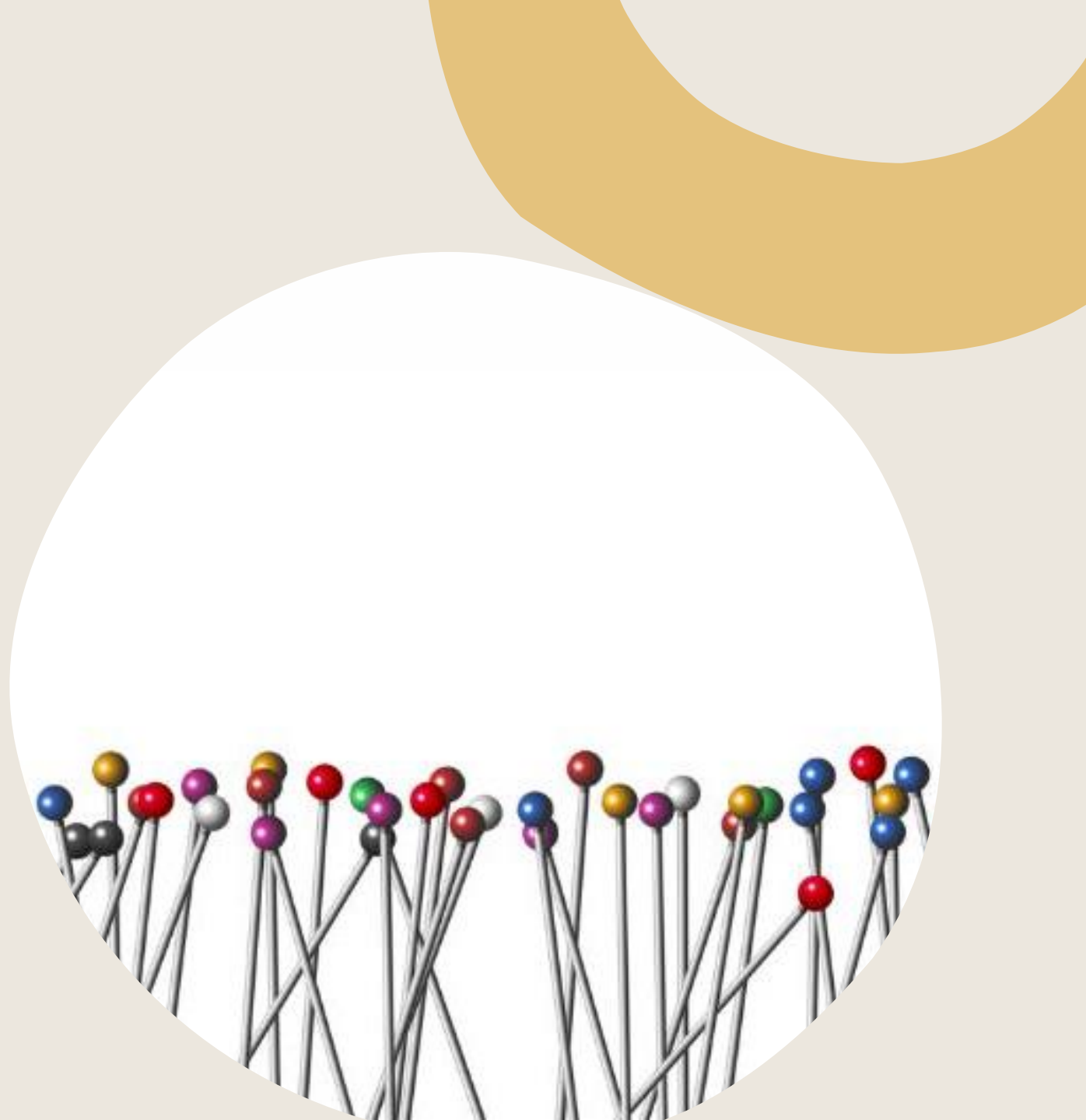
**APORTES DEL TEMA A SU
FORMACION PROFESIONAL Y/O
PERSONAL**

Logro de Aprendizaje

- AL FINALIZAR LA SESION LOS ESTUDIANTES UTILIZAN LOS CONCEPTOS E IDENTIFICAN LOS TIPOS DE ESTRUCTURA DINAMICAS Y SU IMPORTANCIA EN LA PROGRAMACIÓN PARA APLICARLOS EN LA SOLUCION DE EJERCICIOS PRACTICOS.

Estructuras de Datos Dinámicas y Tipos Abstractos de Datos (TAD)

Fundamentos para organizar y gestionar
información eficientemente



Introducción a las Estructuras de Datos Dinámicas

Concepto y relevancia de las estructuras dinámicas

Flexibilidad y Adaptación

Las estructuras dinámicas crecen o reducen durante la ejecución, adaptándose a cambios en tiempo real sin pérdida de eficiencia.

Eficiencia en memoria

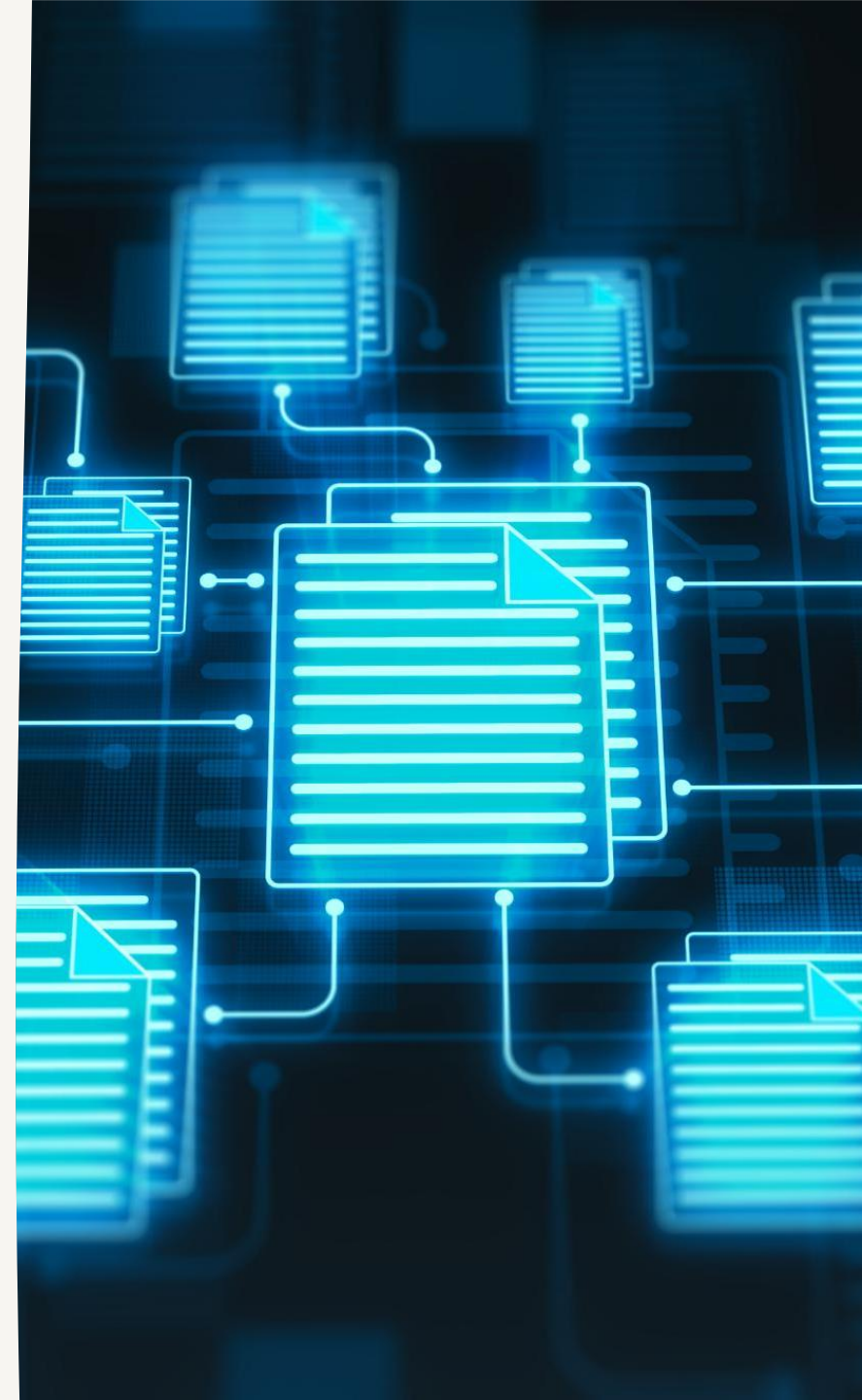
Permiten una administración eficiente de memoria comparadas con arreglos estáticos, mejorando rendimiento y escalabilidad.

Bases para estructuras complejas

Sirven de base para árboles, grafos y colas prioritarias, esenciales en algoritmos de búsqueda y optimización.

Modularidad y mantenimiento

Facilitan soluciones limpias y modulares, reduciendo acoplamiento y separando comportamiento de implementación física.



Tipo Abstracto de Datos (TAD): Fundamentos

Definición y características principales

Modelo Conceptual de TAD

El TAD define valores y operaciones sin detallar la implementación interna, enfocándose en el comportamiento deseado.

Ocultación de Información

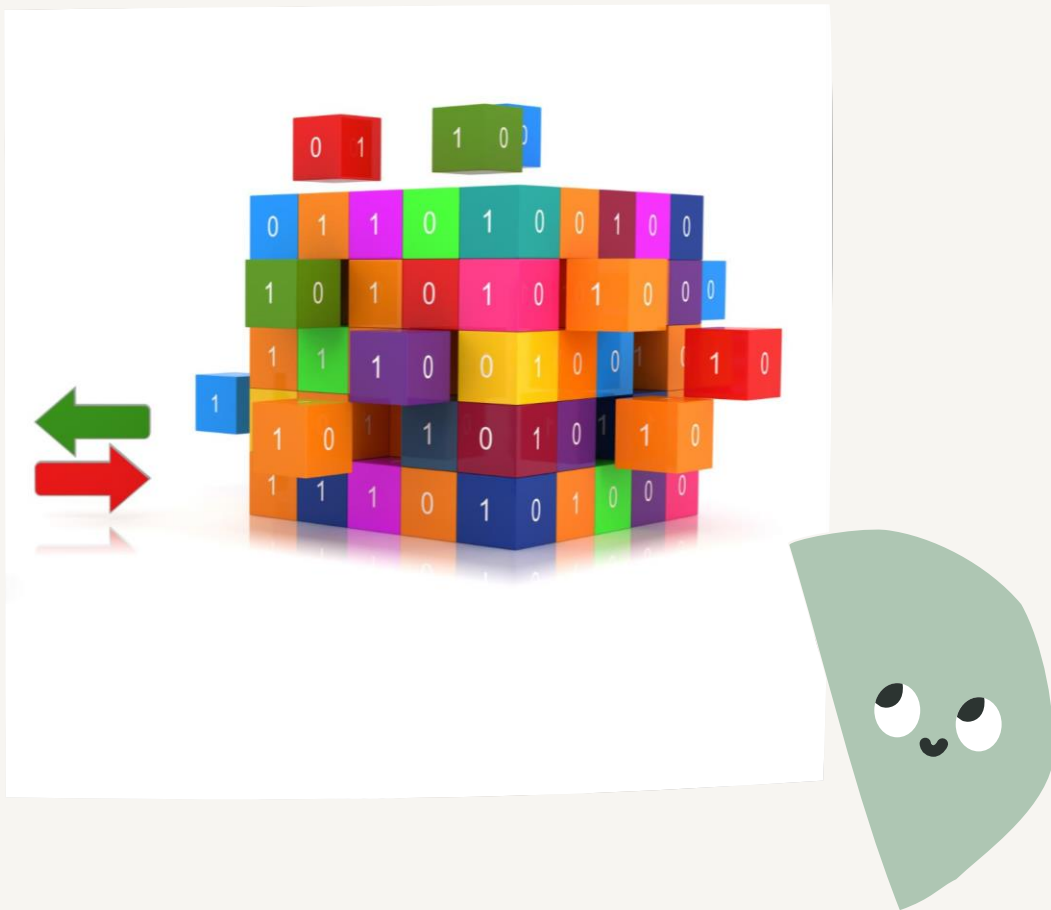
Separa el qué del cómo, protegiendo la estructura interna y facilitando la reutilización y el mantenimiento del código.

Operaciones de un TAD

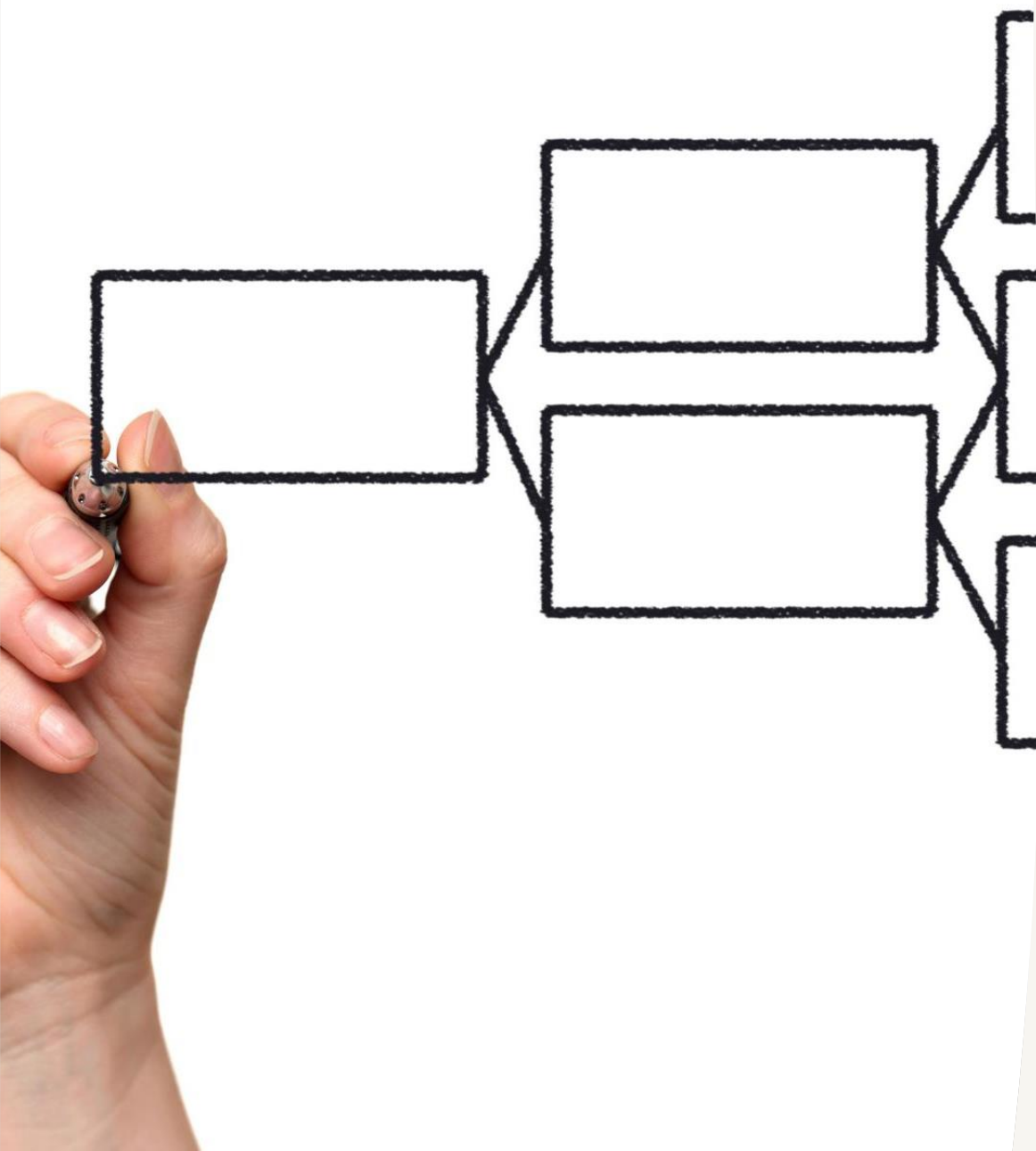
Incluye operaciones constructoras, modificadoras y observadoras, como push, pop y peek en un TAD Pila.

Marco Matemático y Contrato Lógico

Los TAD permiten razonar sobre algoritmos y asegurar invariantes que garantizan consistencia y funcionamiento correcto.



Especificación de un TAD



Especificación informal de un TAD

Comunicación accesible

La especificación informal usa lenguaje natural y ejemplos para facilitar la comprensión de las operaciones del TAD sin formalismos.

Operaciones del TAD Cola

Describe operaciones típicas como enqueue, dequeue y front, y cómo se comportan bajo condiciones normales.

Importancia y uso

Ideal para documentar, enseñar y colaborar, estableciendo expectativas claras antes de especificaciones formales.

Especificación formal de un TAD

Uso de axiomas y estados para definir comportamiento



Especificación formal precisa

El uso de axiomas y lenguaje matemático elimina ambigüedades al definir comportamiento exacto de TADs.

Definición de condiciones y dominios

Se establecen precondiciones, postcondiciones e invariantes que aseguran validez y correcto funcionamiento.

Aplicaciones en software crítico

La formalización garantiza confiabilidad en sistemas financieros, médicos y aeronáuticos, evitando errores graves.

Pruebas y verificación automática

Permite generar modelos verificables que aseguran la implementación respeta el contrato definido por el TAD.

Implementación de un TAD con ejemplos en Java

Ejemplo práctico: implementación de un TAD Pila en Java



Definición de la Interfaz Pila

La interfaz Pila define métodos esenciales para manejar una pila sin especificar su estructura interna.

Implementación con ArrayList

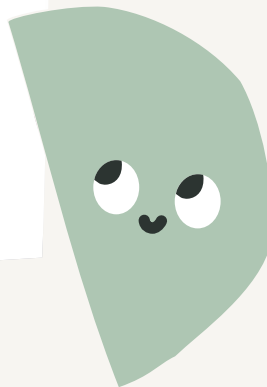
La implementación con ArrayList permite un acceso eficiente al final de la lista para operaciones de pila.

Implementación con Nodos Enlazados

La estructura de nodos enlazados muestra la naturaleza dinámica y flexible de la pila en memoria.

Concepto de TAD y Abstracción

Distintas implementaciones pueden cumplir el TAD si respetan la especificación abstracta del comportamiento.



Solución

Problema: Verificar si una expresión con paréntesis/llaves/corchetes está **balanceada** (e.g., `(([]{}))` es válida; `([])` no).

TAD elegido: Pila (Stack).

Implementación: dinámica con nodos enlazados (sin límite fijo).

Uso: Función `balanceado(String expr)` que emplea la pila.

1) Contrato (especificación informal → formal breve)

TAD `Stack<T>`

• **Observadores:** `isEmpty() : boolean`, `size() : int`, `peek() : T`

• **Modificadores:** `push(T x) : void`, `pop() : T`

• **Axiomas** (informales):

- `isEmpty()` después de crear → `true`.
- `push(x)` seguido de `peek()` → devuelve `x`.
- `push(x)` seguido de `pop()` → devuelve `x` y deja la pila como antes del `push`.
- `size()` aumenta en `+1` con `push` y disminuye en `-1` con `pop` (si no está vacía).

• **Precondiciones:**

- `peek()` y `pop()` **requieren** `!isEmpty()` (si no, lanzar excepción).

2) Código completo en un solo archivo (listo para compilar)

Guarda como `DemoTAD.java` y compila con `javac DemoTAD.java`; ejecuta con `java DemoTAD`.

```
import java.util.NoSuchElementException;
```

```
import java.util.Locale;
```

```
/**
```

```
 * DemoTAD
```

```
 * -----
```

```
 * Ejemplo práctico de TAD Pila (Stack) con implementación dinámica (nodos enlazados)
```

Ejemplo didáctico?

Objetivo didáctico

- Entender qué es un **TAD**: *un contrato (qué hace) sin decir cómo lo hace.*
- Ver **operaciones básicas**: `push`, `pop`, `peek`, `isEmpty`, `size`.
- Separar la **interfaz (contrato)** de la **implementación**.
- Practicar **arrays**, **condicionales** y **bucles**.

Historia (metáfora)

Imaginemos una **pila de platos** en la cocina:

- Cuando llega un plato limpio, lo **ponemos arriba** (`push`).
- Cuando necesitamos un plato, **sacamos el de arriba** (`pop`).
- Si queremos **mirar** cuál está arriba sin sacarlo: `peek`.
- Podemos preguntar si **está vacía** o cuántos platos hay: `isEmpty`, `size`.

Implementación sencilla (para principiantes)

Para que sea **fácil de entender**, implementamos la pila con un **array** interno y un **índice** que apunta al tope. Así evitamos punteros y listas enlazadas por ahora.

2) Clase `ArrayStack<T>` (implementación con array)

```
import java.util.NoSuchElementException;
```

```
// Implementación simple con array (capacidad fija) — fácil para empezar
```

```
public class ArrayStack<T> implements Stack<T> {  
    private Object[] data; // almacenamos como Object y casteamos al regresar  
    private int top;       // apunta al próximo lugar libre (también = tamaño actual)
```

```
    public ArrayStack(int capacidad) {
```

EGERCI



Programa con menú (usando String = "platos")

Ahora un **main con menú** por consola para que el alumno pruebe `push/pop/peek` interactivo.

3) MainPlatos.java (menú de consola)

```
import java.util.Locale;
import java.util.Scanner;
```

```
public class MainPlatos {
    public static void main(String[] args) {
        Locale.setDefault(Locale.US);
        Scanner sc = new Scanner(System.in);
```

```
// 1) Creamos la pila de "platos" (Strings) con capacidad 5
Stack<String> pilaDePlatos = new ArrayStack<>(5);
```

```
// 2) Menú simple
int opcion;
do {
```

```
    System.out.println("\n=== Pila de Platos ===");
    System.out.println("1) Agregar un plato (push)");
    System.out.println("2) Sacar un plato (pop)");
    System.out.println("3) Ver plato de arriba (peek)");
    System.out.println("4) ¿Está vacía?");
    System.out.println("5) ¿Cuántos platos hay? (size)");
    System.out.println("0) Salir");
    System.out.print("Elige: ");
    opcion = leerEntero(sc);
```

```
try {
    switch (opcion) {
        case 1:
            System.out.print("Nombre del plato (p.ej., 'plato azul'): ");
            String plato = sc.nextLine().trim();
            if (plato.isEmpty()) {
                System.out.println("Debe ingresar un nombre.");
            } else {
                pilaDePlatos.push(plato);
                System.out.println("✓ Se agregó: " + plato);
            }
            break;
```

EJERCICIOS

Cómo compilar y ejecutar
1) Guardar 3 archivos en la misma carpeta:
Stack.java, ArrayStack.java, MainPlatos.java

```
javac Stack.java ArrayStack.java MainPlatos.java  
java MainPlatos
```

¿Qué aprende el estudiante con este ejemplo?

1. **TAD como contrato** (`Stack<T>`) → define **qué** operaciones existen.

2. **Implementación intercambiable** (`ArrayStack<T>`) → **cómo** se hacen internamente.

3. **Uso aplicado** (menú) → volver concreta la abstracción con una metáfora: *platos*.

4. **Manejo de errores**: casos de pila vacía y llena.

5. **Tipos genéricos y arrays**.

Luego puedes **evolucionar** este ejemplo: cambiar `ArrayStack<T>` por `LinkedList<T>` (nodos) **sin tocar** el menú; solo cambias la **línea de construcción** y el resto sigue funcionando. Eso demuestra el **poder del TAD**.

 **Variante (si quieres reforzar “memoria limitada”)**

- Deja capacidad chica (por ejemplo, 3) para que vivan el **overflow** y entiendan que con *array* la capacidad es finita.

- Después presentas la **versión dinámica** con **nodos**, donde no hay límite fijo (más avanzada, pero el contrato es el mismo).

EGERCICIOS



Preguntas

- ✚ De ejemplos de estructura de datos estática:
- ✚ De ejemplos de estructura de datos dinámica lineal:
- ✚ De ejemplos de estructura de datos dinámica no lineal:

Desaprende lo que te limita

1. Defina qué es una estructura dinámica?
2. Clasifique ejemplos en estáticas, dinámicas lineales y no lineales.



**Universidad
Tecnológica
del Perú**